Registo de decisões — Assignment #1

Cada decisão de implementação ou de protocolo fica registada aqui, com a justificação, as alternativas consideradas e o que deve aparecer no relatório. O código refere estas entradas como `Dxx` nos comentários.

Legenda de estado: **Implementada** · **Acordada** (decidida, ainda por implementar) · **Pendente** (falta decidir/confirmar)

Decisões importantes / fora do comum (rever se necessário)

Lista das escolhas que **não vêm diretamente do enunciado** ou que **se desviam do procedimento mais óbvio**. Cada uma pode mudar os números do relatório. Para cada uma: o que foi feito, porquê, e **como voltar atrás**. Os detalhes estão na entrada `Dxx` indicada.

#	Decisão fora do comum	Porquê	Como voltar atrás	Ver
1	**Estudo do FAST: threshold 5 excluído** (usam-se 10, 20, 40, 80)	Com t=5 o FAST marca ~15% dos píxeis; são os pontos de menor confiança (orientação do professor: descartá-los), a repetibilidade é quase toda ao acaso e o cálculo bloqueia	Acrescentar `5` a `CFG["fast_study"]["thresholds"]`	D24
2	**Estudo do FAST: repetibilidade não calculada acima de 50 000 pontos por imagem**	O emparelhamento 1-para-1 bloqueou (> 14 min) com 160 mil pontos num só par	Mudar `CFG["fast_study"]["max_points_repeatability"]` (risco de bloquear)	D24
3	**Estudo do FAST: repetibilidade média só nos pares comuns a todas as configurações** (188 de 285 em `i`; 113 de 295 em `v`)	Por causa do ponto 2, cada configuração ficava avaliada em pares diferentes e as médias não eram comparáveis	Em `fast_study.summarize_fast` e `plot_fast`, usar `df` em vez de `common_pairs(df)`	D24
4	**Estudo do FAST: "repetibilidade ao acaso"** medida com a H ground-truth deslocada 20 px	Com muitos pontos a repetibilidade sobe por coincidência; é preciso um nível de referência	Remover a coluna `repeatability_chance`; os deslocamentos estão em `CFG["fast_study"]["chance_shifts_px"]`	D24
5	**Rotação e escala avaliadas num teste sintético** (GRAF imagem 1 rodada/escalada), não no HPatches	O HPatches só separa iluminação e ponto de vista; o professor pediu a discussão das 4 categorias	Não correr `--rotscale`; níveis em `CFG["synthetic"]`	D19, D20
6	**Controlos (0° e escala 1,0) fora das médias** de rotação/escala	São triviais (precisão 1,0) e inflacionariam a média	Em `evaluation.summarize`, não filtrar a coluna `control`	D20
7	**Máscara circular na imagem do teste de rotação**	Sem ela, rodar cria cantos pretos e perde conteúdo, o que gera keypoints falsos	`rot_scale_study.make_pairs`: não aplicar `circular_feather`	D20
8	**GRAF `img4.ppm` chama-se "Imagem 3"** nas figuras, legendas e tabelas	Só há 3 imagens GRAF; numeração 1, 2, 3 no relatório (pedido da Carolina)	`datasets.GRAF_REPORT_NUMBER = {2: 2, 4: 4}`	D21
9	**Correspondências ground-truth por emparelhamento 1-para-1 máximo** (não o método guloso do código de arranque)	Garante N_correct ≤ N_corresp e recall ≤ 1	Trocar o cálculo em `evaluation.gt_correspondences`	D17

#	Decisão fora do comum	Porquê	Como voltar atrás	Ver
10	**Limite de 2000 keypoints por imagem** nas experiências principais (maior confiança primeiro)	Comparação justa entre métodos; o enunciado avisa que mais pontos não significa melhor	`--max-kps none` ou `CFG["max_keypoints"] = None`	D05
11	**Precisão = 0 quando não há matches** (em vez de "não definida")	Um método que não emparelha nada falhou nesse par; ignorar o par inflacionaria a média	`evaluation.evaluate_pair`: devolver NaN	D18
12	**N_features conta todos os keypoints da imagem 1** (não só os da zona comum)	Definição literal de PMR e matching score; o enunciado deixa a escolha aos alunos	`evaluation.evaluate_pair`: usar `N_vis1`	D02
13	**Deteção e descrição em chamadas separadas** para medir os dois tempos	O enunciado pede tempos separados; no SIFT/KAZE a soma fica maior do que um `detectAndCompute`	`descriptors.extract_classic`	D06
14	**Keypoints nas figuras com marcador fixo** (sem escala nem orientação) e **amostra de 150 linhas** nos matches	Os círculos à escala do ORB tapavam a imagem; com todas as linhas não se via nada	`visualization.plot_keypoints`; `CFG["qualitative"]["max_matches_drawn"]`	D21
15	**Métricas da homografia (T4) ainda não existem**	Dependem do `homography.py` da Inês; decidido não implementar nada sobre código por fazer	Implementar quando existir	P05
16	**BRIEF próprio: patch S = 48 px**	Nenhuma das versões do artigo (ECCV 2010, TPAMI 2012) dá um valor para S; 48 é o do BRIEF do OpenCV (margem medida de 28 px)	`CFG["brief_own"]["patch_size"]`	D25
17	**BRIEF próprio: grelha polar da G V com 8 anéis × 32 ângulos**	O artigo só diz "coarse polar grid containing n_d points"	`CFG["brief_own"]["polar_radii"]`	D25
18	**BRIEF próprio: suavização gaussiana 9×9, σ = 2** (ECCV 2010) e não 7×7 ou filtro de caixa (TPAMI 2012)	Segue a versão original e coincide com o kernel 9 do OpenCV	`CFG["brief_own"]["blur_ksize"]`, `["blur_sigma"]`	D25
19	**BRIEF próprio: 4 estratégias (G I, II, III, V); a G IV não foi implementada**	A G IV é a menos especificada no artigo; o enunciado pede pelo menos 2	Acrescentar "GIV" em `brief_own.make_pairs` e em `STRATEGIES`	D25

#	Decisão fora do comum	Porquê	Como voltar atrás	Ver
20	**BRIEF próprio: coordenadas fora do patch encostadas à borda também na G II**	O artigo só o diz para a G III; com σ = S/5 cerca de 1% das coordenadas cai fora	`brief_own.make_pairs`	D25
21	**FAST com threshold 20 nas experiências principais** (a omissão do OpenCV é 10)	Valor herdado do código de arranque do guia; igual ao `fastThreshold` do ORB. Descoberto na verificação de 2026-10-12	`CFG["detectors"]["FAST"]["threshold"] = 10` e repetir as corridas	D09, P09

Protocolo e interface

D01 — Interface comum entre módulos

- **Estado:** Implementada (`src/interface.py`)
- **Decisão:** cada extrator devolve `Features` (kps, desc, norm, t_det, t_desc, method, meta) e cada matcher devolve `MatchResult` (matches, t_match, method, meta). Uma configuração avaliável é um `Pipeline` = extrator + matcher.
- **Regras:** keypoints em coordenadas da imagem original; `desc` float32 (N, D) para descritores float e uint8 empacotado (N, bits/8) para binários; `queryIdx` → imagem 1, `trainIdx` → imagem 2; matches 1-para-1 e putativos (antes do RANSAC); tempos em ms.
- **Justificação:** permite que `evaluation.py` (Pessoa A) avalie os métodos da Pessoa B e que `stitching.py` (Pessoa B) use os da Pessoa A sem mexer no código um do outro. Separar extrator de matcher obriga a distinguir desempenho do detetor/descritor do desempenho do matcher, como pede o enunciado (§2.6, §4). Ex.: "SuperPoint+NN" e "SuperPoint+LightGlue" partilham o extrator.
- **Alternativa:** tuplos soltos (como no `starter.py`). Rejeitada: fácil de trocar a ordem dos campos e não há lugar para metadados (ex.: tensores do LightGlue).

D02 — Denominadores das métricas

- **Estado:** Acordada (implementar em `evaluation.py`)
- **Decisão:** `#N_features` = todos os keypoints da imagem 1 (depois do `compute()`). `#N_correspondences` = keypoints da imagem 1 visíveis na imagem k que, projetados com a H GT, têm um keypoint da imagem k a < 3 px, emparelhados 1-para-1. A repetibilidade usa só os pontos na zona comum: `N_corresp / min(n1_visíveis, nk_visíveis)` .
- **Justificação:** o enunciado (§2.8) deixa a definição aos alunos mas exige que seja reportada. Usar todos os pontos em `N_features` segue a definição literal de PMR e matching score (Heinly et al.). A repetibilidade é uma medida só do detetor e, por isso, só faz sentido na zona onde o ponto pode reaparecer.
- **No relatório:** escrever as duas definições e reportar os denominadores em todas as tabelas.

D03 — Uma única implementação do RANSAC

- **Estado:** Acordada (Pessoa B, `homography.py`)
- **Decisão:** `evaluation.py` usa o `estimate_homography` de `homography.py`. Parâmetros em `CFG["ransac"]`: limiar 3 px, 2000 iterações, confiança 0,995.
- **Justificação:** os inliers e o corner error da Parte 1 e o stitching da Parte 2 ficam calculados exatamente da mesma forma.

D04 — Localização e dimensão do HPatches

- **Estado:** Implementada
- **Decisão:** o zip extraía para `data/hpatches-sequences-release/hpatches-sequences-release/`. A pasta interior subiu um nível; o caminho é `data/hpatches-sequences-release/<seq>/`.
- **Verificado:** 116 sequências (57 `i_*` + 59 `v_*`), ou seja 580 pares 1→k (k = 2…6). As imagens têm tamanhos diferentes entre sequências (ex.: 480×640 em `i_ajuntament`, 640×800 em `v_graffiti`).
- **Decisão:** usar o HPatches completo para os métodos clássicos. Para os aprendidos em CPU decide-se depois (ver P03).

D05 — Orçamento comum de 2000 keypoints

- **Estado:** Implementada (`detectors.cap_keypoints`, `CFG["max_keypoints"]`)
- **Decisão:** todos os detetores ficam limitados aos 2000 pontos com maior `response`, com ordenação estável (resultado determinístico). Com `max_keypoints=None` não há limite.
- **Justificação:** comparação justa (o guia, §6.4, e o enunciado, §5: "um número grande de keypoints não indica um bom método"). Sem limite, o FAST com threshold baixo produz dezenas de milhares de pontos e domina o PMR e os tempos.
- **Nota:** é um teto, não um alvo. Alguns métodos detetam menos (ex.: KAZE: 490 pontos em `i_ajuntament` 1). Isto faz parte do comportamento do detetor e deve ser discutido.
- **Alternativa:** usar o `nfeatures` de cada método. Só existe no SIFT e no ORB, por isso o corte é feito por nós, de forma igual para todos (exceção: ORB, ver D10).
- **Possível extra:** repetir a experiência sem limite e discutir a diferença.
- **Cuidado (verificado em 2026-10-09, D24):** o FAST com NMS **desligada** devolve `response = 0` em todos os pontos. Aí o corte guardaria só os primeiros pontos pela ordem de varrimento (as linhas de cima da imagem). Nas experiências principais o FAST usa sempre NMS ligada, por isso os resultados não são afetados; fica o aviso no código (`detectors.cap_keypoints`).

D06 — Medição de tempos

- **Estado:** Implementada (incluindo o aquecimento, `evaluation._warm_up`)
- **Decisão:** `time.perf_counter()`, em ms. Deteção e descrição são medidas em chamadas separadas (`detect()` e depois `compute()`). A criação do objeto OpenCV não entra no tempo. O corte ao orçamento (D05) entra no tempo de deteção.
- **Justificação:** o enunciado pede tempos separados de deteção, descrição e matching.
- **Consequência a referir no relatório:** no SIFT e no KAZE, `compute()` reconstrói o espaço de escalas, por isso `t_det + t_desc` é maior do que um `detectAndCompute()` único.
- **Aquecimento (observado):** a primeira chamada ao ORB demorou 961 ms e as seguintes ~7 ms (inicialização interna do OpenCV). Por isso, `run_hpatches` corre cada pipeline uma vez no primeiro par, sem medir, antes de começar. Para métodos em GPU: `torch.cuda.synchronize()` antes de cada leitura do tempo (Pessoa B).
- **Como se reportam (`evaluation.evaluate_pair`):** `t_det_ms` e `t_desc_ms` = média por imagem (img1 e imgk); `t_match_ms` = por par (inclui as pesquisas 1→2 e 2→1 do cross-check); `t_total_ms` = 2·(t_det + t_desc) + t_match.
- **Nota:** as features da imagem 1 são calculadas uma vez por sequência e reutilizadas nos 5 pares, por isso o tempo da img1 é o mesmo nas 5 linhas da sequência.

D07 — Seeds fixas

- **Estado:** Implementada (`config.setup_env`)
- **Decisão:** `cv2.setRNGSeed(0)` e `np.random.default_rng(0)` no início de cada experiência.
- **Justificação:** reprodutibilidade (enunciado §7). Afeta o RANSAC e a amostragem de pares do BRIEF próprio.

Implementação

D08 — Leitura e escrita de imagens com caminhos acentuados

- **Estado:** Implementada (`datasets.imread`, `datasets.imwrite`)
- **Decisão:** ler com `np.fromfile` + `cv2.imdecode` e escrever com `cv2.imencode` + `tofile`.
- **Justificação:** no Windows, `cv2.imread` falha sem aviso (devolve `None`) em caminhos com caracteres não-ASCII, e o caminho do projeto tem "º" e "ê". Assim também funciona nos outros sistemas.
- **Regra:** nenhum módulo deve usar `cv2.imread` / `cv2.imwrite` diretamente.

D09 — Parâmetros dos detetores

- **Estado:** Implementada (`CFG["detectors"]`)
- **Decisão:** valores por omissão do OpenCV 4.13, escritos explicitamente no config, **exceto o threshold do FAST**:
  - SIFT: 3 camadas/octava, contrastThreshold 0,04, edgeThreshold 10, σ 1,6 (omissão)
  - ORB: scaleFactor 1,2, 8 níveis, edgeThreshold 31, fastThreshold 20 (omissão)
  - KAZE: threshold 0,001, sem `extended` e sem `upright` (omissão)
  - FAST: NMS ligada e tipo 9/16 (omissão); **threshold 20 (a omissão do OpenCV é 10)**
- **Justificação:** os valores por omissão são os recomendados pelos autores/OpenCV. Afiná-los por método favoreceria uns métodos em relação a outros. O efeito dos parâmetros do FAST é estudado à parte (T6).
- **Correção de 2026-10-12:** esta entrada dizia que o threshold 20 do FAST era o valor por omissão. Não é: a verificação contra a versão instalada (`FastFeatureDetector_create().getThreshold()` = 10) e a documentação mostram que a omissão é 10. O 20 veio do código de arranque do guia da cadeira (`docs/starter.py`) e é igual ao `fastThreshold` por omissão do ORB. Todos os outros parâmetros foram confirmados como iguais à omissão (ver VERIFICACOES.md, secção 2.2).
- **Estado do threshold do FAST:** decisão pendente (P09): manter 20 como escolha documentada ou passar a 10 e repetir as corridas. Com o limite de 2000 pontos (os de maior resposta), a diferença só existe nas imagens em que o FAST com t = 20 encontra menos de 2000 pontos.

D10 — ORB: `nfeatures` = orçamento

- **Estado:** Implementada
- **Decisão:** `ORB_create(nfeatures=max_keypoints)` ; sem orçamento usa-se 100 000.
- **Justificação:** o valor por omissão do ORB é 500, o que o deixaria em desvantagem. O ORB distribui os pontos pelos níveis da pirâmide de acordo com `nfeatures` , por isso o limite tem de ser dado na criação (o corte posterior por si só não chega). O SIFT usa `nfeatures=0` + corte comum.

D11 — Descritores e combinações válidas

- **Estado:** Implementada (`descriptors.py`)
- **Decisões:**
  - SIFT, KAZE e ORB só com o seu próprio detetor (dependem da escala/octava atribuídas por ele); combinações como "FAST+SIFT" dão erro.
  - BRIEF, BRISK e FREAK funcionam sobre keypoints FAST (configurações "FAST+X" do enunciado, §4).
  - BRIEF do OpenCV com 32 bytes (256 bits, valor do artigo) e sem orientação (BRIEF original).
  - BRISK e FREAK com os parâmetros por omissão.
  - `compute()` remove pontos perto da borda: usa-se sempre a lista devolvida. `meta["n_detected"]` guarda o número antes da filtragem (ex.: FAST 737 → BRIEF 583).
  - Se não sobrar nenhum ponto, `desc = None` .
  - Normas: L2 para SIFT/KAZE, Hamming para ORB/BRIEF/BRISK/FREAK.
- **Observação preliminar (1 par, não é resultado):** FAST+BRIEF cai para precisão 0,57 em `v_graffiti` 1→2 e 0,67 em GRAF 1→2, contra > 0,9 nos restantes métodos. É coerente com o BRIEF não ter orientação. Confirmar no HPatches completo e discutir.

D12 — Threads do OpenCV

- **Estado:** Implementada (`CFG["num_threads"]`)
- **Decisão:** por omissão (`None`) o OpenCV usa todas as threads (16 nesta máquina).
- **Justificação:** é a forma normal de uso. Os tempos são reportados indicando o CPU. Se for preciso isolar o custo algorítmico, pode fixar-se `num_threads=1` e repetir.

D16 — Matching: ratio test + cross-check, força bruta

- **Estado:** Implementada (`matching.match_descriptors` , `matching.mutual_nn`)
- **Decisão:** para cada descritor da img1 procuram-se os 2 vizinhos mais próximos na imgk; aceita-se o match se `d1 < 0,8·d2` (ratio de Lowe) **e** se o melhor vizinho da imgk → img1 for o mesmo ponto (cross-check). Resultado: matches 1-para-1. Por omissão usa-se força bruta (`BFMatcher` , exato).
- **Justificação:** ratio 0,8 e emparelhamento 1-para-1 são o protocolo de referência do enunciado (§2.8). A força bruta é exata, por isso as diferenças entre métodos não são confundidas com erros da pesquisa aproximada.
- **Alternativas implementadas:**
  - `CFG["matching"]["backend"] = "flann"` : KD-trees para float, LSH para binários, `checks=50` . Para a comparação força bruta vs aproximado pedida no stitching (§3.1, passo 4).
  - `mutual_nn` : só vizinho mútuo, sem ratio (para descritores aprendidos, onde 0,8 pode ser demasiado restritivo).
- **Detalhe:** o cross-check usa só o 1.º vizinho no sentido 2→1, sem ratio (como o `crossCheck` do OpenCV). Pares com menos de 2 vizinhos são ignorados (o ratio não está definido).

D17 — Correspondências ground-truth e repetibilidade

- **Estado:** Implementada (`evaluation.gt_correspondences`)
- **Decisão:**
  1. Zona comum: pontos da img1 cuja projeção H·p cai dentro da imgk, e pontos da imgk cuja projeção H^{-1}·p cai dentro da img1.
  2. Dois pontos podem corresponder se ||H·p1 − pk|| < 3 px (erro medido na imgk).
  3. `N_corresp` = **emparelhamento 1-para-1 máximo** desse grafo bipartido (`scipy.sparse.csgraph.maximum_bipartite_matching`).
  4. Repetibilidade = `N_corresp / min(n1_visíveis, nk_visíveis)` .
- **Justificação:** com o emparelhamento máximo, qualquer conjunto de matches corretos 1-para-1 cabe em `N_corresp` , o que garante **N_correct ≤ N_corresp** e **recall ≤ 1**. O greedy do `starter.py` (para cada ponto, o vizinho mais próximo; se já usado, desiste) pode subcontar e dar recall > 1.
- **Verificação:** com H = identidade e a mesma imagem, repetibilidade = precisão = recall = 1,0 nos 6 métodos. Num subconjunto de 4 sequências (120 linhas): MS = PMR × precision, recall ≤ 1 e N_correct ≤ N_corresp em todas as linhas.
- **Limitação a referir:** a repetibilidade é calculada com os keypoints depois do `compute()` , por isso FAST+BRIEF/BRISK/FREAK têm valores ligeiramente diferentes (cada descritor remove pontos diferentes da borda). A repetibilidade "pura" do detetor FAST é medida no estudo do FAST (T6, D24).
- **Implementação (desde D24):** os pares a < 3 px são encontrados com uma KD-tree, não com uma matriz densa. O resultado é idêntico (verificado).

D18 — Casos limite e agregação

- **Estado:** Implementada (`evaluation.evaluate_pair` , `evaluation.summarize`)
- **Casos limite:**
  - `N_putative = 0` → precisão 0 (o método falhou nesse par; com NaN a média ficaria inflacionada).
  - `N_corresp = 0` → recall e repetibilidade NaN (não há o que recuperar; excluídos da média).
  - `N_features = 0` → PMR e MS = 0.
- **Agregação:** média por par ("macro"), agrupada por pipeline × categoria (`i` / `v` / `rot` / `scale`) e por pipeline × categoria × k (curvas F1; no sintético `k` = nível). Os pares de controlo (coluna `control`) são excluídos das médias por categoria, mas aparecem nas curvas (D20). As contagens (denominadores) também são médias por par e aparecem ao lado das métricas na T2.
- **Robustez:** um erro num par/pipeline não pára a corrida: fica registado na coluna `error` e na tabela `errors` .

D19 — Robustez discutida em 4 categorias + análise qualitativa

- **Estado:** Implementada (`main.py --robustness` , tabela `<tag>_robustness.csv` com PMR, precision, matching score, recall e repetibilidade em i/v/rot/scale)
- **Decisão:** a robustez é avaliada em 4 categorias, cada uma com média por categoria e por nível:

Categoria	Dados	Nível (coluna `k`)
`i` iluminação	HPatches `i_*` (57 seq)	par 1→k, k = 2…6
`v` ponto de vista	HPatches `v_*` (59 seq)	par 1→k, k = 2…6
`rot` rotação	sintético a partir da GRAF img1 (D20)	ângulo (°)
`scale` escala	sintético a partir da GRAF img1 (D20)	fator de escala

  Mais uma análise qualitativa com o GRAF 1→2 e 1→3 (figuras de matches certos e errados, P08).

- **Justificação:** o professor pediu explicitamente a discussão de escala, rotação, iluminação e ponto de vista (§5 do enunciado). O HPatches só separa `i` e `v` : as `v_*` misturam perspetiva, rotação e escala, por isso não permitem isolar a rotação nem a escala.
- **Porque não usar o GRAF para a rotação/escala:** o GRAF é uma sequência de mudança de ponto de vista (perspetiva forte, com alguma rotação/escala misturadas). Só tem 2 pares e os valores não são controlados, o que não dá médias nem curvas com significado. Serve como exemplo qualitativo, como o enunciado sugere ("particularly for qualitative analysis").
- **Complemento real possível:** `v_bark` e `v_boat` (sequências de zoom + rotação do conjunto de Oxford, incluídas no HPatches), para confirmar em imagens reais o que o sintético mostra.

D20 — Teste sintético de rotação e escala

- **Estado:** Implementada (`studies/rot_scale_study.py` , `CFG["synthetic"]` , `main.py --rotscale`)
- **Imagem base:** GRAF img1 (320×400, textura rica).
- **Níveis:** rotação 0°–180° em passos de 15° (13 níveis); escala 0,5; 0,6; 0,7; 0,8; 0,9; 1,0; 1,25; 1,5; 1,75; 2,0 (10 níveis). 0° e 1,0 são **controlos**: entram nas curvas, mas não nas médias por categoria (seriam triviais e inflacionariam a média).
- **Rotação:** em torno do centro, na mesma tela. Antes de rodar, a imagem base fica só com o círculo inscrito, com transição suave (10 px) para um cinzento uniforme (a média da imagem). Assim a rotação não cria nem perde conteúdo, e a borda não gera keypoints falsos. A img1 do par de rotação é esta imagem mascarada.
- **Escala:** `cv2.resize` em torno do centro, na mesma tela: `INTER_AREA` para reduzir (evita aliasing), `INTER_LINEAR` para ampliar. Na redução, a margem é preenchida com o mesmo cinzento; na ampliação, a imagem é recortada. A H inclui a convenção de centros de pixel do `cv2.resize` (`x' = s·(x + 0,5) − 0,5 + offset`).
- **H ground-truth:** é a própria transformação aplicada (exata).
- **Verificação:** `warpPerspective(img1, H)` comparado com a imagem gerada dá um erro médio de 0,0–0,34 níveis de cinzento (rotação e ampliação) e ≤ 2,3 níveis na redução (diferença entre a suavização do `INTER_AREA` e a interpolação linear, não um erro geométrico).
- **Correção feita:** a 1.ª versão usava fundos com cinzentos ligeiramente diferentes na máscara e na rotação, o que criava arestas ténues nos cantos da imagem rodada. Passou a usar-se um único valor de fundo.
- **Resultados preliminares (só para validar o teste; repetir na corrida oficial):** coerentes com a teoria. FAST+BRIEF: precisão 0,88 a 15°, 0,17 a 30° e 0 a partir de 45° (BRIEF sem orientação). FAST+BRIEF/BRISK/FREAK: 0,02–0,14 a 0,5× e a 2× (FAST sem escala). SIFT e KAZE ≥ 0,79 em todos os níveis. ORB 0,87–0,89 com rotações > 135°.
- **Limitações a referir:** é uma única imagem (planar, textura de graffiti); não há ruído nem mudança de iluminação; a média por categoria depende dos níveis escolhidos (por isso as curvas por nível são o resultado principal).

D21 — Figuras qualitativas

- **Estado:** Implementada (`visualization.py` , `studies/qualitative.py` , `main.py --qualitative` , `CFG["qualitative"]`)
- **Organização: uma figura por método e por objetivo** (mais fácil de colocar no relatório), em `results/figures/qualitative/<objetivo>/<método>.jpg` (`+` → `-` no nome, ex.: `FAST-BRIEF.jpg`):
  - `keypoints/` : keypoints na GRAF imagem 1.
  - `graf_1to2/` , `graf_1to3/` : matches; **verde = correto** (erro < 3 px com a H GT), **vermelho = errado**.
- **Numeração GRAF no relatório (decisão de 2026-10-08):** só temos 3 imagens GRAF, numeradas 1, 2, 3 no relatório: imagem 1 = `img1.ppm` , imagem 2 = `img2.ppm` , **imagem 3 = `img4.ppm`** (homografia `H1to4p.txt`). O código lê `img4.ppm` ; só os rótulos usam "3" (`datasets.GRAF_REPORT_NUMBER` , `datasets.load_graf_pairs`). O cabeçalho do `legendas.md` repete esta correspondência.
  - `rot45/` , `scale2/` : o mesmo para os pares sintéticos de rotação 45° e escala 2×.
  - `legendas.md` : legenda pronta para cada figura, gerada com os números da corrida (protocolo de matching, totais, linhas mostradas e, no FAST+X, quantos pontos o descritor descartou junto à borda).
- **Conteúdo de cada figura:** eixos x/y em píxeis; título com o método e os números (ex.: "KAZE — GRAF 1→3: 27 de 52 matches corretos (52 %)"); cada imagem identificada ("Imagem 1", "Imagem 3", "Imagem 2 (rodada 45°)"); legenda das cores. Na figura de matches, o eixo y da imagem da direita fica à direita, para as linhas não o taparem.
- **Keypoints:** desenham-se **todos**, com um marcador de tamanho fixo igual para todos os métodos. A 1.ª versão usava a escala do keypoint (`DRAW_RICH_KEYPOINTS`) e só os 500 mais fortes: no ORB isso dava círculos enormes (patch de 31 px × fator da pirâmide) concentrados no centro, o que tornava a comparação ilegível. Com o marcador fixo comparam-se número e distribuição espacial; a escala e a orientação não são representadas (dito na legenda).
- **Matches:** amostra aleatória (seed fixa) de até 150 linhas, que mantém a proporção certos/errados; os totais estão sempre no título e na legenda; os errados são desenhados por cima.
- **Formato:** JPEG, qualidade 92, 150 dpi (≈ 13 MB no total para 30 figuras; em PNG eram 39 MB).
- **Reutilização:** `plot_matches(..., good=mask, good_label=..., bad_label=...)` aceita qualquer máscara booleana. A Pessoa B pode usá-la para inliers/outliers do RANSAC no stitching (§3.1, passo 8).
- **Observações para o relatório (corrida de 2026-10-08):**
  - GRAF 1→3 (ponto de vista forte): SIFT 49%, KAZE 52%, FAST+FREAK 47%, FAST+BRISK 38%, ORB 32%, FAST+BRIEF 0% (13 matches, todos errados).
  - Rotação 45°: SIFT 100%, KAZE/BRISK/FREAK 99%, ORB 96%, FAST+BRIEF 0%.
  - Distribuição espacial: o SIFT espalha os pontos por toda a imagem; o ORB concentra-os nas zonas de alto contraste do centro e não tem pontos a menos de ~31 px da borda (`edgeThreshold`). O score de Harris favorece essas zonas e o ORB não força uma distribuição uniforme.

D22 — Corrida preliminar dos clássicos (2026-10-08)

- **Estado:** Feita (resultados em `results/tables/classic_*`, fora do git por agora)
- **Comandos:** `--part1`, `--rotscale` e `--robustness`, todos com `--tag classic`. Parâmetros por omissão do config (2000 kps, ratio 0,8 + cross-check, 3 px, força bruta, 16 threads, i7-12650H).
- **Execução:** 3480 linhas HPatches (580 pares × 6 métodos) em 913 s; 0 erros. Sintético: 23 níveis × 6.
- **É preliminar:** a corrida oficial repete-se no fim com os métodos aprendidos e a T4 (P05), numa só execução, para os tempos serem comparáveis.
- **Observações para o relatório (números desta corrida):**
  - Precisão i / v / rot / scale: SIFT 0,74 / 0,75 / 0,99 / 0,97; KAZE 0,78 / 0,74 / 0,99 / 0,94; ORB 0,70 / 0,69 / 0,92 / 0,94; FAST+BRIEF 0,78 / 0,60 / 0,09 / 0,57; FAST+BRISK 0,74 / 0,55 / 0,99 / 0,50; FAST+FREAK 0,68 / 0,54 / 0,99 / 0,55.
  - FAST+BRIEF é dos melhores em iluminação (sem rotação/escala) e o pior em rotação: falta de orientação. FAST+X cai em escala (o FAST não tem escala) e em ponto de vista.
  - SIFT: o recall mais alto em `v` (0,51) e em escala (0,90).
  - Custo total por par: FAST+BRIEF 23 ms, ORB 41 ms, FAST+BRISK 39 ms, FAST+FREAK 60 ms, SIFT 336 ms, KAZE 1291 ms. Memória dos descritores por imagem: SIFT 878 kB, KAZE 393 kB, ORB 61 kB, FAST+BRIEF 52 kB.
- **Suspeita resolvida:** no teste com 4 sequências, a precisão em `i_*` (0,4–0,7) parecia baixa. No total fica em 0,68–0,78. A média é puxada para baixo por algumas sequências extremas (precisão média sobre os 6 métodos: `i_pool` 0,10, `i_tools` 0,21, `i_londonbridge` 0,23, `i_brooklyn` 0,28), enquanto as melhores passam de 0,95. É um efeito dos dados, não do código. Vale a pena analisar essas sequências na discussão.

D23 — Métricas nos pares GRAF (1→2 e 1→3)

- **Estado:** Implementada (`main.py --graf`, `datasets.load_graf_pairs`) e corrida em 2026-10-08
- **Decisão:** os 2 pares GRAF são avaliados com o mesmo protocolo do HPatches (mesmas métricas, 3 px, ratio 0,8 + cross-check). Como são só 2 pares, **não se fazem médias**: a tabela `<tag>_graf_pairs.csv` mostra cada par (método × par) com os denominadores, PMR, precision, MS, recall, repetibilidade e tempos. Não entra na tabela de robustez (D19), que usa médias.
- **Justificação:** a utilizadora quis números para acompanhar as figuras qualitativas (D21). Os números das figuras e da tabela coincidem (mesma corrida).
- **Observações (corrida de 2026-10-08):**
  - 1→2: precisão ≥ 0,94 em todos os métodos exceto FAST+BRIEF (0,67, só 51 matches).
  - 1→3: a precisão cai para 0–0,52 (SIFT 0,49; KAZE 0,52; FREAK 0,47; BRISK 0,38; ORB 0,32; BRIEF 0), mas a **repetibilidade mantém-se alta (0,54–0,74)**. Os detetores voltam a encontrar os mesmos pontos; o que falha é o descritor, porque a perspetiva forte deforma a vizinhança de cada ponto. Exemplo claro para separar o desempenho do detetor do desempenho do descritor.

D24 — Estudo do detetor FAST (threshold × NMS, comparação com SIFT)

- **Estado:** Implementada (`studies/fast_study.py` , `CFG["fast_study"]` , `main.py --fast`)
- **Pedido:** enunciado §2.3: "investigate the effect of detector parameters, such as the intensity threshold and non-maximum suppression, and compare FAST against SIFT in terms of repeatability, number of detected points and runtime".
- **Desenho:**
  - Configurações: threshold ∈ {10, 20, 40, 80} × NMS {ligada, desligada}, tipo 9/16 → 8 configurações do FAST, mais o SIFT com os parâmetros por omissão como referência. (O t=5 foi excluído em 2026-10-09; ver "Confiança dos keypoints" abaixo.)
  - Dados: HPatches completo (580 pares 1→k), separado por `i` e `v` .
  - Só o **detetor**: sem descritor, por isso a repetibilidade não é afetada pelo `compute()` (resolve a limitação da D17).
  - **Sem limite de pontos** (`max_keypoints=None`) em todas as configurações e no SIFT: o objetivo é ver o comportamento real do detetor, incluindo a explosão do nº de pontos com threshold baixo.
  - Métricas por par: nº de pontos (média img1/imgk), pontos na zona comum, correspondências GT (D17), repetibilidade, **repetibilidade ao acaso** e tempo de deteção por imagem (média img1/imgk, com aquecimento, D06).
- **Repetibilidade ao acaso (porquê e como):** com muitos pontos, quase todos têm um vizinho a < 3 px por coincidência, e a repetibilidade sobe sem mérito do detetor. Exemplo: FAST t=5 sem NMS em `v_graffiti` 1→2 teve 64 mil / 68 mil pontos e repetibilidade 0,93. Para separar o efeito real do acaso, calcula-se a mesma medida com a H ground-truth **deslocada 20 px** (média de (+20, +20) e (−20, −20) px): mantém o nº e a distribuição real dos pontos, mas quebra a correspondência geométrica.
  - Alternativa rejeitada: a fórmula para pontos uniformes, $1 - \exp(-\pi \cdot \varepsilon^2 \cdot \text{densidade})$. No teste dava valores **acima** da repetibilidade medida no FAST sem NMS (ex.: 0,92 ao acaso vs 0,62 medida em `i` , t=5). Os pontos sem NMS formam aglomerados, que cobrem muito menos área do que pontos uniformes, por isso a fórmula sobrestima o acaso.
  - Limitação: 20 px é uma escolha (≫ 3 px, mas ainda dentro da zona com textura). É uma estimativa do nível ao acaso, não um valor exato.
- **Alteração de suporte:** `evaluation.gt_correspondences` passou a usar uma KD-tree (`scipy.spatial.cKDTree.sparse_distance_matrix`) em vez da matriz de distâncias densa. A densa, com 64 mil × 68 mil pontos, precisaria de ~35 GB. Verificado: N_corresp idêntico à versão densa em 100 casos (5 sequências × 5 pares × 4 métodos); 64 mil pontos em 0,2 s.
- **Saídas:** `<tag>_fast_raw.csv` (uma linha por configuração × par), `<tag>_fast_T6.csv` (tabela T6: configuração × categoria), figuras `results/figures/fast_study/fast_<métrica>_<i|v>.png` (repetibilidade, nº de pontos, tempo; uma por métrica e categoria) e `legendas.md` .

Incidente (2026-10-08/09): a corrida bloqueou na sequência 2

- **Sintoma:** a 1.ª corrida completa ficou mais de 14 min parada em `i_autannes` (2.ª sequência), a consumir CPU.
- **Diagnóstico:**
  - a KD-tree e o emparelhamento eram rápidos nos pares 1→2 … 1→5 (≤ 0,3 s, mesmo com 160 mil pontos);
  - o bloqueio estava no par **1→6**, configuração **t=5 sem NMS**: 160 mil × 163 mil pontos e 1,05 milhões de ligações (pares a < 3 px). O `maximum_bipartite_matching` do scipy (Hopcroft–Karp) não terminou em 4 min;
  - causa: os pontos estão tão densos que as ligações se encadeiam pela imagem toda, e **264 mil dos 323 mil pontos formam um único grupo ligado** (componente). Por isso dividir o grafo em componentes também não ajuda.
- **Ordem de grandeza:** 160 mil keypoints numa imagem de 870×1280 são ~15% de todos os píxeis. Já não são pontos de interesse.

Confiança dos keypoints e exclusão do t=5

- **Teoria:** cada keypoint tem um score de confiança (`KeyPoint.response`). No FAST, o score é o **maior threshold para o qual o píxel ainda seria detetado**, isto é, quanto o arco de 9 píxeis contíguos do círculo é mais claro/escuro do que o centro. Pontos de score baixo estão mesmo no limiar: pouco repetíveis (ruído, luz e perspetiva fazem-nos desaparecer), pouco distintivos e redundantes.
- **Orientação do professor:** os pontos de baixa confiança podem ser descartados, porque acrescentam pouco.
- **No FAST, descartar por confiança = subir o threshold:** um ponto com score s é detetado para qualquer t ≤ s, por isso "FAST t=20" = "FAST t=5, guardando só os pontos com confiança ≥ 20". A **NMS** é outra forma de usar a confiança: em cada grupo de vizinhos fica só o de score máximo.
- **Verificado (GRAF img1):** com NMS ligada, `response` vai do threshold até 195 (t=5: mínimo 5, mediana 14; t=20: mínimo 20, mediana 37). Com **NMS desligada,** `response = 0` **em todos os pontos**: o OpenCV só calcula o score quando precisa dele para a supressão. Sem NMS não há confiança para ordenar ou filtrar; o único filtro é o próprio threshold.
- **Decisão:** excluir o t=5. É o corte de confiança mais baixo, marca ~15% dos píxeis como "cantos", a sua repetibilidade é quase toda ao acaso (no teste: 0,93 medida contra 0,88 ao acaso em `v`) e torna o cálculo impraticável.

Rede de segurança: limite de pontos para a repetibilidade

- **Decisão:** se uma das imagens do par tiver mais de **50 000 pontos** (`CFG["fast_study"]["max_points_repeatability"]`), a repetibilidade (e o nível ao acaso) desse par **não é calculada**: fica NaN, a coluna `rep_skipped` = True, e é excluída da média. O nº de pontos e o tempo continuam a ser medidos. A T6 mostra `n_rep_skipped` e a legenda dos gráficos de repetibilidade indica quantos pares foram excluídos em cada configuração.
- **Justificação:** garante que a corrida não bloqueia em nenhuma configuração/par extremo, sem ter de adivinhar quais. 50 mil pontos por imagem é muito acima do útil (o SIFT dá ~3–4 mil).
- **Teste:** `i_autannes` + `v_graffiti` correm em 6 s. Em `i_autannes` , o t=10 sem NMS (~82 mil pontos/imagem) ficou com os 5 pares excluídos e o t=20 sem NMS com 2.
- **Consequência:** ver a subsecção seguinte.

Médias comparáveis: só os pares comuns a todas as configurações (2026-10-10)

- **Problema (corrida completa de 2026-10-09):** a rede de segurança excluiu muito mais pares nas configurações que geram mais pontos:

Configuração	pares excluídos em `i` (de 285)	em `v` (de 295)
t=10, NMS ligada	4	11
t=20, NMS ligada	0	5
t=10, NMS desligada	97	182
t=20, NMS desligada	41	86
t=40, NMS desligada	0	5
t=40 e t=80 com NMS, t=80 sem NMS, SIFT	0	0

  Os pares excluídos não são aleatórios: são os das imagens com mais textura. Cada configuração ficava com a repetibilidade média calculada num conjunto de pares diferente, e os valores não eram comparáveis.

- **Exemplo para perceber:** é como comparar a média de dois alunos quando um fez os 10 testes e o outro faltou aos 4 mais difíceis. A média do segundo não diz que ele é melhor.

- **Decisão:** a repetibilidade (medida e ao acaso) é a média só sobre os **pares em que todas as configurações, incluindo o SIFT, foram avaliadas**: 188 de 285 em `i` e 113 de 295 em `v` (`fast_study.common_pairs`). O nº de pontos e o tempo continuam a ser médias sobre os 580 pares, porque essas medidas existem sempre.

- **Na T6:** `n_pairs` (todos), `n_pairs_rep` (comuns) e `n_rep_skipped` (excluídos nessa configuração). As legendas dos gráficos de repetibilidade dizem quantos pares entram na média.

- **Alternativas rejeitadas:**
  - manter as médias sobre pares diferentes e só indicar os excluídos: comparação injusta;
  - excluir também o t=10 sem NMS: mais uma exclusão, e o t=20 sem NMS continuava com o problema;
  - subir o limite de pontos: o bloqueio ocorreu com 160 mil, não se sabe a partir de onde é seguro.

- **Limitação a referir no relatório:** a repetibilidade do estudo do FAST não inclui as imagens com mais textura (ex.: `i_autannes`), sobretudo em `v` (113 de 295 pares). As conclusões valem para esse subconjunto.

- **Como refazer sem repetir a corrida:** `python src/main.py --fast --tag classic --reuse-raw` (lê `classic_fast_raw.csv` e refaz a T6, as figuras e as legendas).

Resultados (corrida de 2026-10-09: 116 sequências, 407 s; médias refeitas em 2026-10-10)

Configuração	pontos/img `i`	pontos/img `v`	t_det `i` (ms)	t_det `v` (ms)	rep. `i` (acaso)	rep. `v` (acaso)
FAST t=10, NMS ligada	11 971	18 984	2,2	3,6	0,67 (0,36)	0,70 (0,38)
FAST t=20, NMS ligada	5 675	9 527	1,2	2,1	0,64 (0,23)	0,66 (0,20)
FAST t=40, NMS ligada	2 038	3 745	0,6	1,0	0,61 (0,13)	0,66 (0,09)
FAST t=80, NMS ligada	485	885	0,3	0,5	0,53 (0,05)	0,60 (0,05)
FAST t=10, NMS desligada	45 877	75 744	2,5	4,2	0,78 (0,39)	0,80 (0,39)
FAST t=20, NMS desligada	19 181	32 906	1,2	2,2	0,72 (0,25)	0,75 (0,20)
FAST t=40, NMS desligada	6 107	10 736	0,6	1,0	0,65 (0,14)	0,70 (0,10)
FAST t=80, NMS desligada	1 232	2 112	0,3	0,4	0,54 (0,05)	0,62 (0,05)
SIFT (referência)	3 425	6 196	77,7	100,6	0,47 (0,14)	0,52 (0,15)

Pontos e tempo: 285 pares `i`, 295 pares `v`. Repetibilidade: 188 pares `i`, 113 pares `v` (comuns).

- **Observações para o relatório:**
  - **Threshold:** subir o threshold reduz muito o nº de pontos (21–25× de t=10 para t=80, com NMS) e reduz pouco a repetibilidade (0,67 → 0,53 em `i`). O nível ao acaso cai muito mais (0,36 → 0,05): com threshold alto, a repetibilidade que sobra é quase toda real.
  - **NMS:** desligá-la dá 2,5–4× mais pontos e mais 0,04–0,12 de repetibilidade em t ≤ 40, mas os pontos a mais são vizinhos redundantes dos mesmos cantos. Com t=80 a diferença quase desaparece (0,53 vs 0,54 em `i`; 0,60 vs 0,62 em `v`). Sem NMS os pontos não têm score de confiança.
  - **FAST vs SIFT:** o FAST é 24–270× mais rápido (0,3–4,2 ms contra 78–101 ms por imagem). Com um nº de pontos da mesma ordem (t=40 com NMS: 2–4 mil; SIFT: 3–6 mil), o FAST tem repetibilidade mais alta nos dois casos (0,61 vs 0,47 em `i`; 0,66 vs 0,52 em `v`), com nível ao acaso semelhante (0,09–0,13 vs 0,14–0,15). Ressalva: o SIFT deteta em várias escalas e a repetibilidade aqui só mede a posição (3 px); a invariância à escala do SIFT não é avaliada neste estudo (ver D20).

D25 — BRIEF próprio (4 estratégias de amostragem)

- **Estado:** Implementada (`src/brief_own.py` , `CFG["brief_own"]` , `studies/brief_study.py` , `main.py --brief`)
- **Pedido:** enunciado §2.5: "Students are encouraged to implement a simplified BRIEF descriptor themselves, following the sampling strategies described in the original paper. At least two sampling strategies should be investigated."
- **Fontes lidas (2026-10-10/11):**
  - M. Calonder, V. Lepetit, C. Strecha, P. Fua, "BRIEF: Binary Robust Independent Elementary Features", ECCV 2010, secções 3, 3.1 e 3.2 (DOI 10.1007/978-3-642-15561-1_56);
  - M. Calonder et al., "BRIEF: Computing a Local Binary Descriptor Very Fast", IEEE TPAMI 34(7), 2012 (DOI 10.1109/TPAMI.2011.222), a referência do enunciado. PDFs na pasta do trabalho.
- **O algoritmo (artigo, secção 3):** num patch S × S suavizado, centrado no keypoint, fazem-se n_d testes: bit i = 1 se $p(x_i) < p(y_i)$, senão 0. O descritor é a sequência dos bits. As únicas escolhas são a suavização e a posição dos pares (x_i, y_i).
- **As 5 estratégias do artigo (secção 3.2, origem no centro do patch):**

	Pares (X, Y)	Implementada?
G I	X, Y ~ Uniforme(−S/2, S/2)	Sim
G II	X, Y ~ Gaussiana(0, $S^2/25$)	Sim (a usada pelos autores)
G III	X ~ Gaussiana(0, $S^2/25$); Y ~ Gaussiana(X, $S^2/100$); fora do patch → borda	Sim
G IV	X, Y sorteados numa grelha polar grosseira	**Não**
G V	X = (0, 0); Y em todos os pontos de uma grelha polar com n_d pontos	Sim

  Resultado do artigo (Fig. 3): a G V perde para todas as aleatórias; a G II tem pequena vantagem. A versão de 2012 acrescenta que a G V corresponde à transformada Census.
- **Porquê estas 4:** G I, II e III estão totalmente especificadas. A G V é a única determinística e, segundo o artigo, a pior: serve de contraste e de validação (se a nossa implementação reproduzir a ordem do artigo, é sinal de que está correta). A G IV é a menos especificada (só "coarse polar grid") e ficou de fora.
- **Parâmetros (`CFG["brief_own"]`) e origem de cada um:**

Parâmetro	Valor	Origem
n_d (nº de testes)	256 (BRIEF-32)	Artigo: descritor por omissão; igual ao do OpenCV que usamos
Suavização	gaussiana 9×9, σ = 2	ECCV 2010 §3.1 (“we use a value of 2”, janela 9×9)
Patch S	48 px	**Escolha nossa** (o artigo não dá valor)
Grelha polar (G V)	8 anéis × 32 ângulos	**Escolha nossa** (o artigo não a especifica)
Seed	0 (`CFG["seed"]`)	Reprodutibilidade

- **Ambiguidades do artigo e como foram resolvidas:**
  - **Valor de S:** nenhuma das duas versões o indica. Medimos a margem do BRIEF do OpenCV: remove os keypoints a menos de **28 px** da borda = 48/2 + 9/2, ou seja, patch 48 e kernel 9. Usamos S = 48 para a comparação com o OpenCV ser direta (mesmos keypoints sobrevivem).
  - **“Value of 2” na suavização:** o texto fala em variância, mas a legenda da Fig. 1 usa σ (σ = 0,65 … 3,05). Interpretámos como σ = 2.
  - **Janela 9×9 ou 7×7:** o ECCV 2010 diz 9×9; o TPAMI 2012 diz 7×7 e mostra que um filtro de caixa 7×7 dá a mesma exatidão. Usamos 9×9 gaussiana (versão original). O OpenCV usa filtro de caixa com imagens integrais, por isso os descritores não são bit a bit iguais aos nossos.
  - **Coordenadas fora do patch:** o artigo só refere o encosto à borda na G III. Aplicamos o mesmo na G II (com σ = S/5, ±2,5σ é a borda; ~1% das coordenadas cai fora).
  - **Arredondamento:** as coordenadas sorteadas são reais e arredondam-se ao píxel mais próximo.
- **Implementação:**
  - `make_pairs(strategy, S, n_bits, seed, polar_radii)` : gera os pares uma vez (com cache). São os **mesmos pares nas duas imagens** (a mesma seed); sem isso o descritor não é comparável.
  - `BriefOwn.compute(gray, kps)` : suaviza a imagem em float32 (evita empates criados pelo arredondamento para inteiros), remove os keypoints a menos de 28 px da borda, compara os píxeis de todos os pares de uma vez (NumPy) e empacota os bits com `np.packbits` → `uint8 (N, 32)` , usado com `NORM_HAMMING` .
  - Mesma interface do OpenCV (`compute`), por isso entra em `descriptors.describe` sem casos especiais. Pipelines: `FAST+BRIEF-GI` , `FAST+BRIEF-GII` , `FAST+BRIEF-GIII` , `FAST+BRIEF-GV` .
  - Não entram no `--part1` por omissão (`methods.DEFAULT_PIPELINES` = 6 configurações clássicas).
  - Sem orientação nem escala, como o BRIEF original.
- **Propriedades dos pares gerados (seed 0):**

	comprimento médio do par	testes distintos	degenerados (x = y)
G I	24,3 px	256	0
G II	15,9 px	255	0
G III	5,5 px	256	3
G V	13,5 px	240	0

Os 3 testes degenerados da G III (os dois pontos caem no mesmo píxel depois de arredondar) dão sempre bit 0, e os 16 repetidos da G V (ângulos do anel interior que caem no mesmo píxel) dão bits iguais. Não foram corrigidos: são consequência direta do método do artigo e não afetam a comparação (bits constantes ou repetidos contam igual nas duas imagens).

- **Validação (2026-10-11), para evitar o erro do grupo do ano passado (precisão ~1%):**
  - mesma imagem contra si própria: precisão 1,0 e distância de Hamming 0 nas 4 estratégias;
  - nº de keypoints depois do `compute()` igual ao do OpenCV (1455, 1850, 1394 em 3 imagens): a margem à borda é a mesma;
  - pares fáceis: `i_nescafe` 1→2: precisão 0,95–0,99 (OpenCV 0,98); `v_wall` 1→2: 0,91–0,97 (OpenCV 0,95). A G II fica muito próxima do OpenCV (0,976 vs 0,976; 0,957 vs 0,953).
- **Estudo (`--brief`):** FAST + BRIEF do OpenCV e FAST + as 4 estratégias, com o mesmo detetor (FAST t=20, NMS, limite de 2000 pontos), o mesmo matching e o mesmo protocolo, nas 4 categorias (i, v no HPatches completo; rot, scale no teste sintético). Saídas: `<tag>_brief_T7.csv` (tabela T7), `<tag>_brief_T2_matching.csv` (denominadores), `<tag>_brief_T5_cost.csv`, `<tag>_brief_F1_by_k.csv`, `<tag>_brief_raw.csv`; figuras dos padrões em `results/figures/brief_own/brief_pattern_<estratégia>.png` e `legendas.md`.
- **Resultados (corrida de 2026-10-11: 580 pares HPatches + 21 níveis sintéticos, 0 erros):**

Descritor (com FAST)	precisão `i`	precisão `v`	MS `i`	MS `v`	recall `i`	recall `v`	precisão `rot`	precisão `scale`	t_desc (ms)
BRIEF OpenCV	0,777	0,602	0,257	0,125	0,450	0,252	0,091	0,570	4,1
Próprio G I	0,815	0,579	0,272	0,094	0,476	0,187	0,060	0,477	15,2
Próprio G II	0,814	0,632	0,278	**0,130**	0,490	**0,261**	0,109	**0,624**	15,2
Próprio G III	**0,862**	**0,647**	**0,313**	0,115	**0,560**	0,228	0,095	0,592	15,3
Próprio G V	0,710	0,497	0,225	0,095	0,392	0,190	0,111	0,526	14,9

  285 pares `i`, 295 pares `v`, 12 níveis de rotação, 9 de escala. Todos com os mesmos keypoints FAST (N_features médio: 1582 em `i`, 1770 em `v`) e 32 bytes por descritor.
- **Observações para o relatório:**

- **A implementação está validada:** a G II (a do artigo) fica ao nível do BRIEF do OpenCV e um pouco acima em todas as métricas (ex.: MS 0,278 vs 0,257 em `i`; 0,130 vs 0,125 em `v`). A diferença pode vir da suavização (gaussiana nossa, filtro de caixa no OpenCV) e do padrão de pares.
- **Reproduz a conclusão do artigo:** a G V (regular, a partir do centro) é a pior em precisão, matching score e recall em `i`, e em precisão em `v`. Em ponto de vista (o caso testado no artigo, sequência Wall), a G II tem o melhor matching score e recall.
- **G III (pares curtos):** a melhor em iluminação (precisão 0,862; MS 0,313) e com a precisão mais alta em `v`, mas com menos matches em `v` do que a G II (MS 0,115 vs 0,130).
- **G I (pares longos, uniformes):** boa em iluminação, mas a pior em `v` (MS 0,094), em escala (0,477) e em rotação (precisão 0,72 a 15°, contra 0,87–0,90 nas outras). Pares mais longos deslocam-se mais quando a imagem roda ou muda de escala.
- **Rotação:** nenhuma estratégia resolve a falta de orientação. Todas caem a partir de 30° (precisão ≤ 0,22), como no artigo (“up to 10 to 15 degrees, there is little degradation followed by a precipitous drop”). A média da categoria `rot` é ≤ 0,11 em todas.
- **Custo:** o descritor próprio demora ~15 ms por imagem contra ~4 ms do OpenCV (NumPy com suavização gaussiana da imagem inteira vs C++ com imagens integrais). O tamanho é igual (32 bytes).

D26 — Gráficos F1 (curvas por nível) e F2 (exatidão vs tempo)

- **Estado:** Implementada (`studies/curves.py` , `visualization.plot_curves` / `plot_scatter` , `main.py --curves`)
- **Pedido:** o enunciado não pede gráficos específicos, mas pede a discussão da robustez às 4 transformações e do compromisso exatidão vs eficiência (§5). Os gráficos sustentam essas discussões (sugestões F1 e F2 do guia).
- **Dados:** os CSV por par já calculados; **nenhuma corrida é repetida**.
  - Clássicos: `<tag>_raw.csv` + `<tag>_rotscale_raw.csv` → `results/figures/curves/`
  - BRIEF próprio: `<tag>_brief_raw.csv` → `results/figures/curves_brief/`
- **F1 — curvas por nível:** uma figura por categoria (`i` , `v` , `rot` , `scale`) e por métrica (precisão, matching score, recall, repetibilidade), com uma linha por método → 16 figuras (`F1_<métrica>_<categoria>.png`). Eixo x: par 1→k (HPatches), ângulo (rotação) ou fator de escala (eixo logarítmico, para 0,5× e 2× ficarem simétricos em relação a 1).
- **F2 — compromisso:** tempo total médio por par (eixo x, logarítmico) contra matching score e contra precisão (eixo y), um ponto por método, para `i` e `v` → 4 figuras (`F2_<métrica>_vs_time_<categoria>.png`). Tempo total = 2 × (deteção + descrição) + matching (D06).
- **Escolhas de apresentação:**
  - aqui os métodos vão **todos na mesma figura** (ao contrário das figuras qualitativas, D21), porque o objetivo é compará-los;
  - cada método tem a mesma cor e o mesmo marcador em todas as figuras;
  - precisão e repetibilidade com o eixo y fixo de 0 a 1 (figuras comparáveis entre si); matching score e recall com o eixo ajustado aos dados, porque os valores são baixos (< 0,35 em `v`) e com 0–1 as curvas ficavam ilegíveis;
  - os controlos (0° e escala 1) **aparecem** nas curvas (D20: só saem das médias por categoria);
  - no F2 os métodos são identificados por legenda (etiquetas junto aos pontos sobrepunham-se).
- **Rigor nas legendas (`legendas.md` em cada pasta):** nas `v_*` a mudança de ponto de vista aumenta de 1→2 para 1→6; nas `i_*` as imagens 2 a 6 são só condições de iluminação diferentes, **sem ordem de dificuldade garantida**, e a legenda di-lo. Cada ponto das curvas HPatches é a média de 57 (`i`) ou 59 (`v`) pares; nas sintéticas é um único par por nível.
- **Observações para o relatório (corrida de 2026-10-08):**

- **Ponto de vista (precisão, 1→2 … 1→6):** SIFT 0,89 → 0,59; KAZE 0,89 → 0,60; ORB 0,89 → 0,49; FAST+BRIEF 0,78 → 0,41; FAST+BRISK 0,73 → 0,35; FAST+FREAK 0,71 → 0,36. Todos degradam; SIFT e KAZE degradam menos.
- **Rotação:** o FAST+BRIEF cai de 0,88 (15°) para 0,17 (30°) e 0 a partir de 45°; o ORB desce lentamente até 0,87–0,89 acima de 135°; os restantes ficam ≥ 0,97.
- **Escala:** os FAST+X formam um "sino" em torno de 1 (≥ 0,94 entre 0,8× e 1,25×; ≤ 0,35 a 0,6× e ≤ 0,27 a 1,75×); SIFT ≥ 0,95 em todos os níveis; KAZE e ORB descem para 0,79 e 0,85 a 2×.
- **Compromisso (tempo total por par; matching score):**

Método	tempo `i` (ms)	MS `i`	tempo `v` (ms)	MS `v`
FAST+BRIEF	20	0,257	26	0,125
FAST+BRISK	35	0,180	43	0,079
ORB	37	0,154	46	0,133
FAST+FREAK	56	0,169	64	0,086
SIFT	295	0,191	377	0,200
KAZE	1153	0,226	1425	0,173

  Em iluminação, o FAST+BRIEF é ao mesmo tempo o mais rápido e o de maior matching score. Em ponto de vista, o SIFT tem o maior matching score e o KAZE é ~4× mais lento com um valor mais baixo. Entre os rápidos (< 70 ms), o ORB é o melhor em ponto de vista. Base para a discussão "tempo real vs exatidão" pedida no enunciado (a completar com os métodos aprendidos da Pessoa B).

Ambiente e repositório

D13 — Ambiente Python

- **Estado:** Implementada
- **Decisão:** desenvolvimento no conda env `cvc`: Python 3.14.7, opencv-contrib-python 4.13.0.92 (`xfeatures2d` e `KAZE_create` verificados), torch 2.14.1+cu126. CPU: Intel Core i7-12650H (16 threads).
- **`requirements.txt`:** mínimo (numpy, scipy, matplotlib, pandas, psutil, opencv-contrib). Nunca `opencv-python` (partilha o módulo `cv2` e estraga o contrib). torch e LightGlue são acrescentados pela Pessoa B.
- **pandas:** instalado no `cvc` em 2026-10-08 (versão 3.0.6) para `evaluation.py`; o OpenCV ficou intacto (só opencv-contrib-python 4.13.0.92).
- **Incidente (2026-10-08, 11:26):** ao instalar o LightGlue no `cvc`, o pip trouxe `opencv-python` 5.0.0.93, que se sobrepôs ao contrib (`cv2.__version__` passou a 5.0.0 e `cv2.KAZE_create` deixou de existir). Corrigido com `pip uninstall -y opencv-python opencv-contrib-python` + `pip install opencv-contrib-python==4.13.0.92`. Depois disso o LightGlue continua a importar. **Regra:** depois de instalar qualquer pacote, confirmar com `python -c "import cv2; print(cv2.__version__, hasattr(cv2,'KAZE_create'))"` → `4.13.0 True`. Aviso a registar: o torch indica que `torch.jit.script` não é suportado em Python 3.14 (só um aviso por agora; a Pessoa B deve confirmar que o LightGlue corre).

D14 — SURF não avaliado

- **Estado:** Implementada (verificado)
- **Decisão:** o SURF não é avaliado. `cv2.xfeatures2d.SURF_create()` dá `error: (-213: The function/feature is not implemented)` no opencv-contrib-python 4.13.0.92 (algoritmo non-free, desativado nas wheels do pip).
- **No relatório:** "SURF não avaliado: indisponível em opencv-contrib-python 4.13.0.92 (non-free)". O enunciado (§2.2) diz que é opcional.

D15 — Dados e pesos fora do git

- **Estado:** Implementada (`.gitignore`)
- **Decisão:** ficam fora do git `data/` , `*.pth` , `__pycache__/` , `.ipynb_checkpoints/` , `venv/` e `.venv/` . O README explica como obter o HPatches.
- **Justificação:** enunciado §7 ("não incluir datasets grandes nem pesos").

Pendentes / a confirmar

- **P01** — ~~Perguntar ao professor se o BRIEF próprio é obrigatório e com quantas estratégias~~ Resolvido em 2026-10-11: implementadas 4 estratégias (G I, II, III, V), acima do mínimo de 2 do enunciado (D25).
- **P02** — Que imagens usar no stitching (há `data/Panorama/keble_*.jpg` e `data/vehicle-seqs/`).
- **P03** — HPatches completo ou subconjunto para SuperPoint/LightGlue em CPU (a GPU com CUDA parece disponível no `cvc` : torch cu126).
- **P04** — Data de entrega, língua e limite de páginas do relatório.
- **P05 (2026-10-08) — Métricas da homografia por implementar** em `evaluation.py` : inliers do RANSAC, rácio de inliers, erro médio de reprojeção, corner error e % de pares com erro ≤ 1/3/5 px (tabela T4). Dependem de `homography.estimate_homography` (Pessoa B), que ainda não existe. Quando existir: chamá-la em `evaluate_pair` , acrescentar as colunas e a T4 no `summarize` , e registar os limiares no config.
- **P06 (2026-10-08) — Pipelines aprendidos por registar** em `methods.py` (SuperPoint+NN, SuperPoint+LightGlue, SIFT+LightGlue): depende de `learned.py` (Pessoa B).
- **P07 (2026-10-08, atualizado 2026-10-11)** — `main.py` : existem `--part1` , `--rotscale` , `--robustness` , `--qualitative` , `--graf` , `--fast` , `--brief` e `--curves` . Falta `--part2` (Pessoa B).
- **P09 (2026-10-12)** — Threshold do FAST nas experiências principais: manter 20 (escolha documentada) ou usar a omissão do OpenCV, 10, e repetir as corridas (~25 min). Ver D09.
- **P08 (2026-10-08)** — ~~Análise qualitativa GRAF~~ feita (D21). Falta decidir se as figuras (30 JPEG, ≈ 13 MB) vão para o git ou se se geram só para a entrega.